Low-Level Design (LLD) Document

Project Title: *Shipment Pricing Prediction*
Domain: *Supply Chain Analytics*
Difficulty Level: *Intermediate*
Cloud Provider: *AWS*

Contents:

1. **Introduction**
2. **What is LowLevel Design Document?**
3. **Scope**
4. **Architecture | Tools & Technologies Used**
   4.1. Backend Pipeline
   4.2. Frontend Interface
   4.3. Cloud Infrastructure
   4.4. Tools & Technologies Used
5. **Detailed Folder & File Structure**
   5.1. Root-Level Structure
   5.2. src/Shipment_Price_Prediction/
6. **Architecture Description**
7. **Data Description**
   7.1. Data Ingestion
   7.2. Data Validation
   7.3. Data Transformation
   7.4. Model Training
   7.5. Model Evaluation
   7.6. Model Pusher
8. **Data from User**
9. **User Data Validation**
10. **Prediction Pipeline Execution**
11. **Price Prediction & Result Display**
12. **Deployment**
13. **Unit Test Cases**

1. Introduction

The Low-Level Design (LLD) for the **Shipment Pricing Prediction** system provides an in-depth, technical description of how the project is implemented from a programming and infrastructure perspective. While the High-Level Design (HLD) describes the broad functional modules and system interactions, the LLD breaks them down into individual components, files, functions, and configurations. It ensures that every developer working on the system can understand exactly how each part of the project is structured, how it interacts with other parts, and the precise sequence of execution from start to finish. This document serves as a blueprint that bridges conceptual architecture with actual implementation, enabling direct coding without ambiguity.

---

2. What is a Low-Level Design Document?

A Low-Level Design (LLD) document is a **component-level blueprint** that provides the internal logic, class structures, database schema interactions, file purposes, and step-by-step process flows for implementing a system. Unlike the HLD, which focuses on “what” the system will do, the LLD focuses on “how” the system will achieve it, providing precise technical details. In this project, the LLD explains how data is read from MongoDB, processed through various machine learning stages (validation, transformation, training, evaluation), stored in AWS S3, and deployed via FastAPI for prediction through a web interface. It also includes CI/CD automation, containerization, and cloud infrastructure management.

```
SHIPMENT_PRICE_PREDICTION/
│
├── .dvc/                  # DVC internal tracking config
├── .github/workflows/     # CI/CD pipelines
│   └── cicd_workflow.yaml # Automates training & deployment
├── Artifacts/             # Outputs from pipeline stages
│   ├── Data_Ingestion_Artifacts/
│   ├── Data_Transformation_Artifacts/
│   ├── Data_Validation_Artifacts/
│   ├── Model_Trainer_Artifacts/
│   └── S3_Best_Model_Artifacts/
├── config/                # Configuration files
│   ├── schema.yaml        # Data validation rules
│   └── model.yaml         # Model parameters
├── Dataset/               # Raw dataset
├── Documents/             # Project docs
├── Flowcharts/            # Architecture diagrams
├── logs/                  # Training/prediction logs
├── Notebook/              # EDA & testing notebooks
│   ├── Analyzing_Data.ipynb
│   ├── Prediction_notebook.ipynb
│   └── Storing_data_to_MongoDB.ipynb
├── ref_images/            # Reference visuals
├── shipment_env/          # Python virtual environment (3.9)
├── src/                   # Main application code
│   ├── components/        # Each ML pipeline stage
│   ├── configuration/     # S3 & DB operations
│   ├── constant/          # Fixed values
│   ├── entity/            # Artifact/config data classes
│   ├── exception/         # Custom exceptions
│   ├── logger/            # Logging module
│   ├── pipelines/         # Orchestration scripts
│   └── utils/             # Helper functions
├── static/                # CSS/images for UI
├── templates/             # HTML pages for UI
├── .dockerignore          # Ignore files from Docker image
├── .dvcignore             # Ignore files for DVC
├── .env                   # Secrets (AWS, DB)
├── .gitignore             # Ignore files for Git
├── app.py                 # Main FastAPI app
├── Dockerfile             # Container build instructions
├── dvc.yaml               # DVC pipeline definition
├── dvc.lock               # DVC pipeline state
├── LICENSE
├── README.md              # Project description
├── requirements.txt       # Dependencies
├── setup.py               # Package installation script
├── template.py            # Script to recreate structure
├── trial_app.py           # Test script
├── trial_app_runner.py    # Test runner
```

---

3. Scope

The scope of this LLD includes **the complete technical design** of the Shipment Pricing Prediction system, covering the ingestion of shipment data from a MongoDB database, performing quality checks through schema validation, preparing it for modeling via transformation steps, training machine learning models, selecting the best model based on performance metrics, storing it in AWS S3, and deploying it as a fully functional web-based application. Additionally, the scope covers the CI/CD automation pipeline for retraining and redeployment, logging mechanisms for monitoring, and integration with external services like DVC for dataset versioning and MLflow/Dagshub for experiment tracking.

---

4. Architecture | Tools & Technologies Used

The architecture consists of:

- **Backend MLOps Pipeline** (Data ingestion → validation → transformation → training → evaluation → deployment).
- **Frontend UI** (FastAPI with HTML templates).
- **Cloud Storage** (AWS S3 for model artifacts).
- **Version Control** (GitHub + DVC).

4.1 Backend Pipeline

The backend follows an **MLOps architecture** where the workflow is modularized into sequential stages — data ingestion, data validation, data transformation, model training, model evaluation, and model deployment — each implemented in separate Python scripts under a common pipeline orchestration layer.

4.2 Frontend Interface

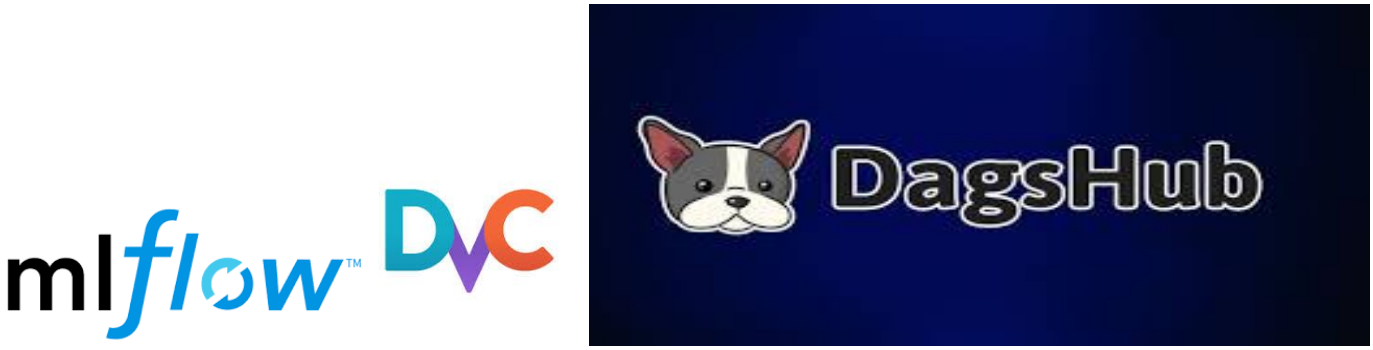
The user interface is built using **FastAPI** for backend API endpoints and HTML templates (with embedded CSS and static files) to present forms, dashboards, and prediction results to the user.

4.3 Cloud Infrastructure

The system integrates with **AWS S3** for storage of trained models, **GitHub Actions** for CI/CD automation, and optionally AWS EC2 or other cloud compute instances for deployment. **DVC** is used for dataset versioning to ensure reproducibility across different stages of model development.

4.4 Tools & Technologies Used


git
GitHub Actions


mlflow
DVC
DagsHub

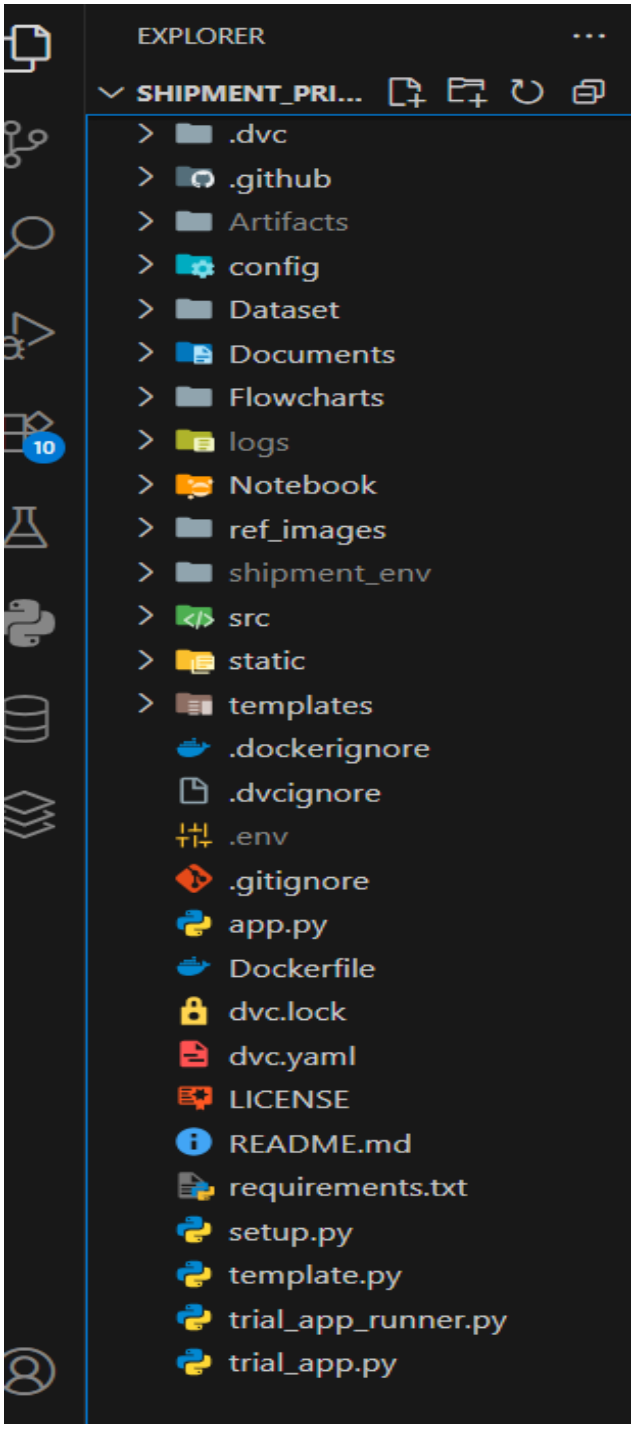

docker
FastAPI
HTML
CSS

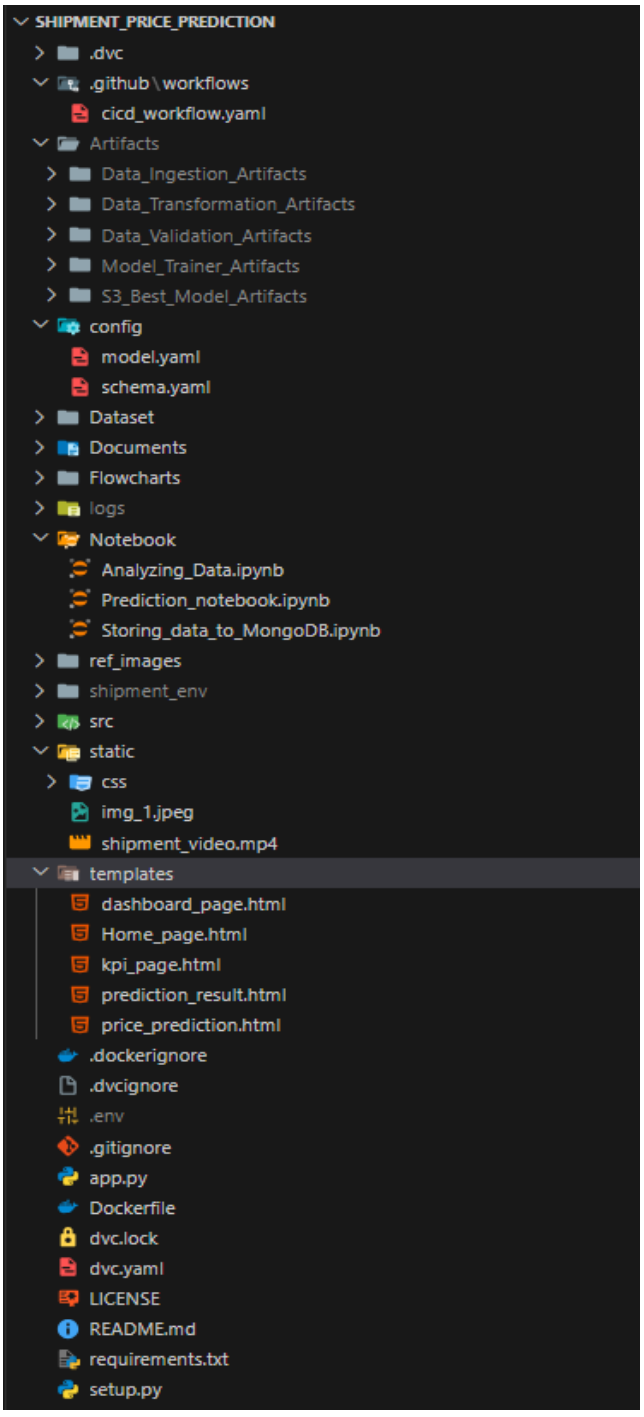

amazon EC2
YAML

---

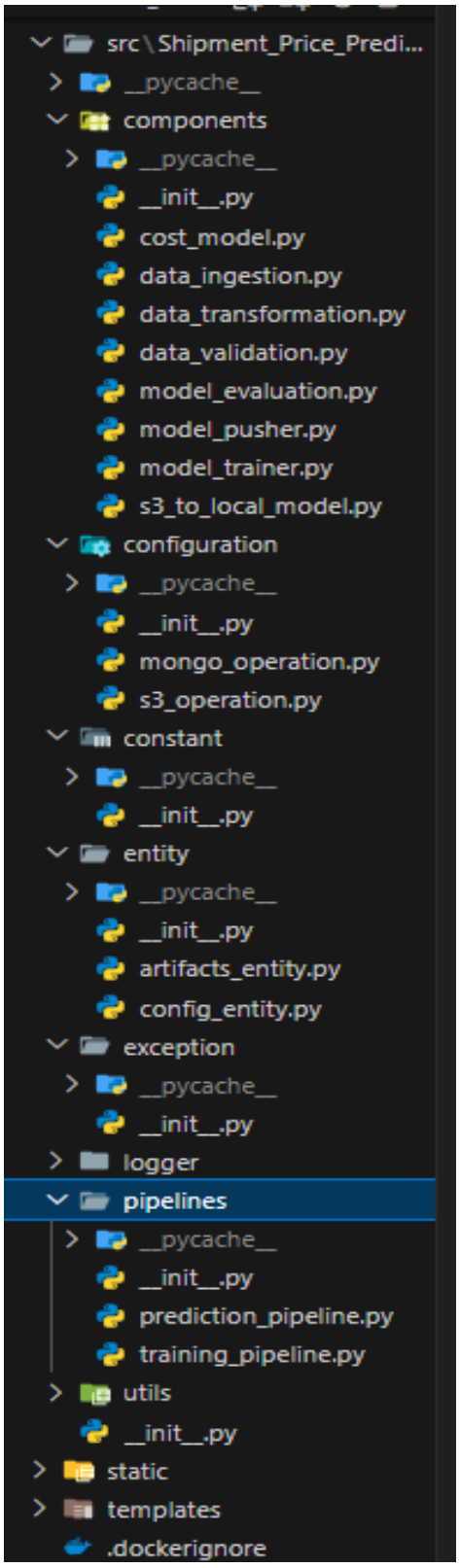
5. Detailed Folder & File Structure

5.1. Root-Level Structure





5.2. src/Shipment_Price_Prediction/



Folder & File Structure Explanation

(a.1) Root-Level Structure

.dvc/

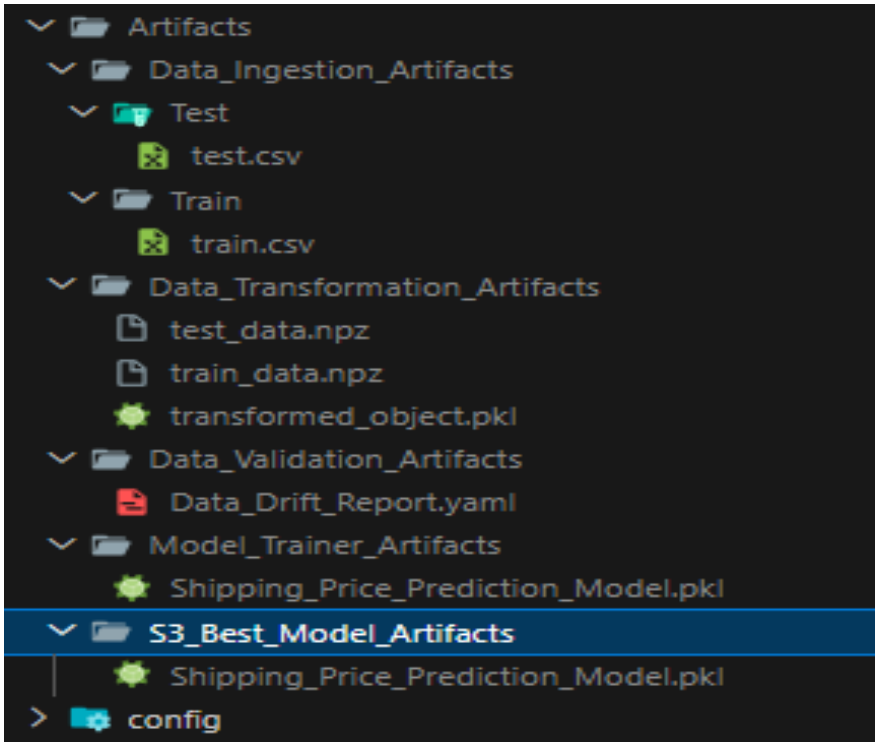
- **Purpose:** Stores Data Version Control (DVC) metadata.
- **Details:** Maintains version history of datasets, intermediate artifacts, and trained models to ensure reproducibility.

.github/workflows/cicd_workflow.yaml

- **Purpose:** Defines the GitHub Actions CI/CD workflow.
- **Details:**
    - Automatically triggers on commits or pull requests.
    - Runs the training pipeline, evaluates models, builds Docker images, and deploys to AWS.

Artifacts/

- **Purpose:** Stores outputs generated from each pipeline stage.
- **Subfolders:**
    - **Data_Ingestion_Artifacts/** – Contains processed training and testing datasets.
    - **Data_Transformation_Artifacts/** – Stores preprocessed data files and transformation objects.
    - **Data_Validation_Artifacts/** – Holds schema validation results and drift reports.
    - **Model_Trainer_Artifacts/** – Contains trained model files and logs.
    - **S3_Best_Model_Artifacts/** – Holds the best-performing model for production use.



config/

- **Purpose:** Central location for project configuration.
- **Files:**
    - schema.yaml – Defines expected dataset structure and validation rules.
    - model.yaml – Contains model parameters, hyperparameters, and algorithm configurations.

Dataset/

- **Purpose:** Stores raw dataset files locally before ingestion.

Documents/

- **Purpose:** Contains all documentation (HLD, LLD, design notes).

Flowcharts/

- **Purpose:** Contains pipeline diagrams and architectural flowcharts.

logs/

- **Purpose:** Stores execution logs for both training and prediction processes.

Notebook/

- **Purpose:** Jupyter Notebooks for experimentation and analysis.
- **Notebooks:**
    - Analyzing_Data.ipynb – Exploratory Data Analysis and visualizations.
    - Prediction_notebook.ipynb – Model testing and initial predictions.
    - Storing_data_to_MongoDB.ipynb – Demonstrates loading data into MongoDB.

ref_images/

- **Purpose:** Contains reference images used in documentation and presentations.

static/

- **Purpose:** Stores static assets for the FastAPI frontend.
- **Includes:** CSS files, images, and demo videos.

templates/

- **Purpose:** HTML templates for the FastAPI web application.
- **Templates:**
    - Home_page.html – Landing page.
    - dashboard_page.html – KPI visualizations.
    - kpi_page.html – Displays business metrics.
    - prediction_result.html – Displays model predictions.
    - price_prediction.html – User input form for shipment details.

Execution & Config Files:

- **app.py** – Main FastAPI application entry point.
- **Dockerfile** – Defines containerization process.
- **requirements.txt** – Lists Python dependencies.
- **setup.py** – Makes the project installable as a package.
- **.env** – Stores environment variables such as API keys and credentials.

(a.2) src/Shipment_Price_Prediction/ – Core Application Code

a) components/ – Pipeline Stage Modules

1. **data_ingestion.py**
    - Loads raw data from local or cloud sources.
    - Splits data into training and testing sets.
    - Saves processed datasets as artifacts.
2. **data_validation.py**
    - Validates dataset structure against schema.yaml.
    - Detects anomalies, missing values, and schema mismatches.
    - Generates validation and drift reports.
3. **data_transformation.py**
    - Transforms features for machine learning readiness.
    - Encodes categorical data and scales numerical features.
    - Stores transformation pipeline for inference.
4. **model_trainer.py**

- Trains multiple ML models using training data.
- Performs hyperparameter tuning.
- Saves the trained model as an artifact.

5. **model_evaluation.py**
   - Compares newly trained models with existing best model .
   - Logs performance metrics to MLflow/Dagshub.
   - Decides if the model should be pushed to production.
6. **model_pusher.py**
   - Pushes the best-performing model to AWS S3 for deployment.
7. **s3_to_local_model.py**
   - Downloads model from AWS S3 to local storage for prediction usage.
8. **cost_model.py** *(optional)*
   - Implements a simple rule-based cost calculation for baseline reference.

---

b) configuration/ – Cloud & Database Management

- **mongo_operation.py** – Manages MongoDB read/write operations.
- **s3_operation.py** – Handles AWS S3 file uploads, downloads, and management.

---

c) constant/

- Stores reusable constant values such as file paths, URLs, and configuration keys.

---

d) entity/

- **artifacts_entity.py** – Defines structured objects to store artifact paths and metadata.
- **config_entity.py** – Defines configuration objects for each pipeline stage.

---

e) exception/

- Custom exception handling for improved debugging and error reporting.

---

f) logger/

- Configures logging with timestamps, levels (INFO, DEBUG, ERROR), and structured log messages.

---

g) pipelines/ – Workflow Orchestration

- **training_pipeline.py** - `Executes complete training workflow (ingestion → validation → transformation → training → evaluation → pushing model).`
- **prediction_pipeline.py** – Handles prediction requests by loading models and transformers, processing input, and returning results.

---

h) utils/

- Contains helper utilities for file I/O, YAML parsing, and common operations used across the project.

(a.3) Component Interactions (LLD ↔ HLD Mapping)

HLD Component	LLD Implementation

Data Ingestion	components/data_ingestion.py
Data Validation	components/data_validation.py
Data Transformation	components/data_transformation.py
Model Training	components/model_trainer.py
Model Evaluation	components/model_evaluation.py
Model Deployment	components/model_pusher.py, pipelines/prediction_pipeline.py, app.py

---

(a.4) Execution Flow

Training Pipeline (**training_pipeline.py**)

1. Ingests data from source.
2. Validates data structure and quality.
3. Transforms features for model readiness.
4. Trains machine learning models.
5. Evaluates against the existing best model.
6. Pushes best model to AWS S3.

Prediction Pipeline (**prediction_pipeline.py** + **app.py**)

1. Loads latest deployed model and transformer.
2. Processes new shipment details from user input.
3. Generates shipment price prediction.

4. Displays results via prediction_result.html.

---

5. Key Interactions

- **FastAPI** ↔ **Prediction Pipeline** – Manages API requests and serves predictions to the frontend.
- **Pipelines** ↔ **AWS S3** – Stores and retrieves model artifacts.
- **Pipelines** ↔ **DVC** – Maintains data and model versioning.
- **Pipelines** ↔ **MLflow + Dagshub** – Tracks experiments and metrics.
- **Data Validation** ↔ **schema.yaml** – Ensures input data meets required format before processing.

---

6. Architecture Description

The architecture starts with the **data ingestion component**, which connects to MongoDB and retrieves the shipment dataset. This data is then stored locally as artifacts, split into training and testing sets, and passed to the **data validation module**, which checks for schema compliance, missing values, and data drift. Validated data is fed into the **data transformation stage**, where categorical features are encoded, numerical features are scaled, and the transformation pipeline is stored for later inference. The transformed data is then processed by the **model training stage**, which evaluates multiple algorithms with hyperparameter tuning. The **model evaluation stage** compares the newly trained model with the previously stored best model and selects the better one based on metrics such as RMSE, MAE, or R^2. The selected best model is then pushed to AWS S3 using the **model pusher** component. For serving predictions, the **prediction pipeline** loads the latest model from S3, processes user input, generates predictions, and returns them through the FastAPI application.

Flow:

- **Data Source:** Shipment data stored in MongoDB.
- **Data Ingestion:** Read and split into train/test sets.
- **Data Validation:** Schema check, missing values, drift detection.
- **Data Transformation:** Scaling, encoding, feature engineering.
- **Model Training:** Multiple models tested with hyperparameter tuning.
- **Model Evaluation:** Best model selected using metrics.
- **Model Pusher:** Store model in AWS S3.
- **Prediction Pipeline:** Load model and make predictions from UI inputs.
- **Deployment:** CI/CD triggers for retraining and redeployment.

---

7. Data Description

7.1 Data Ingestion

Data ingestion connects to the MongoDB database and retrieves raw shipment records. This step includes reading the dataset into a Pandas DataFrame, splitting it into training and test subsets, and saving these subsets as structured artifacts for downstream processing.

7.2 Data Validation

The data validation module checks the incoming dataset against predefined rules stored in schema.yaml. It verifies that all required columns are present, checks for missing values, ensures correct data types, and runs data drift detection to identify shifts in distribution. Any issues are logged, and validation reports are generated.

7.3 Data Transformation

In this step, numerical columns are standardized using scaling techniques, categorical columns are encoded into numeric representations, and any feature engineering steps (such as combining columns or generating new features) are applied. The transformation pipeline is serialized and stored for use during prediction to ensure consistency.

7.4 Model Training

Multiple machine learning algorithms, such as Random Forest, XGBoost, and Linear Regression, are trained on the transformed training set. Hyperparameter tuning is performed using grid search or randomized search to identify the optimal configuration.

7.5 Model Evaluation

The evaluation step loads the previously best-performing model from S3 and compares it with the newly trained one using evaluation metrics. If the new model shows an improvement beyond a certain threshold, it is promoted to production.

7.6 Model Pusher

This component uploads the best-performing model and its associated transformation pipeline to AWS S3, making it accessible for deployment and future predictions.

---

8. Data from User

The system collects shipment details from the user through a web form presented on price_prediction.html. This form includes fields for shipment attributes such as package weight, dimensions, origin, destination, and other cost-impacting parameters.

---

9. User Data Validation

Once the user submits the form, the input is validated to ensure all required fields are filled, data types are correct, and values are within acceptable ranges. Invalid submissions trigger appropriate error messages to guide the user.

- Collected through **price_prediction.html** form in the web app.
- Includes shipment details like weight, origin, destination, etc.

---

10. Prediction Pipeline Execution

The prediction pipeline begins by loading the latest model and transformation pipeline from AWS S3. The submitted user input is transformed to match the model's expected feature format, and the trained model is used to compute the shipment cost prediction. The prediction is then passed to the frontend for display.

---

11. Price Prediction & Result Display

The prediction result is rendered in prediction_result.html, along with the option to return to the prediction form for new entries. This ensures a smooth user experience and supports repeated predictions without reloading the application.

---

12. Deployment

The deployment process is automated through GitHub Actions, which trigger on code commits or pull requests. The CI/CD workflow executes the training pipeline, updates the model if necessary, builds a Docker image of the FastAPI application with the latest model, and deploys it to AWS infrastructure.

- **CI/CD:** GitHub Actions workflow (.github/workflows/cicd_workflow.yaml).
- **Containerization:** Dockerfile for FastAPI + model.
- **Cloud Deployment:** AWS (EC2 or similar service).

---

13. Unit Test Cases

Test Case ID	Test Description	Pre-Requisite	Expected Result
TC-01	Verify that the Home page loads successfully	Application deployed and URL accessible	Home page should load without errors
TC-02	Verify navigation from Home to Prediction page	Home page loaded	Clicking “Prediction” opens the Prediction page
TC-03	Verify navigation from Home to Dashboard page	Home page loaded	Clicking “Dashboard” opens the Dashboard page
TC-04	Verify navigation from Home to KPI page	Home page loaded	Clicking “KPI” opens the KPI page
TC-05	Verify “Make Prediction” button works	Prediction page loaded and form filled	Clicking button redirects to Result page
TC-06	Verify prediction result is correct	Valid inputs submitted	Result page shows correct shipment price prediction
TC-07	Verify “Make Another Prediction” button works	Result page loaded	Clicking returns to Prediction page
TC-08	Verify “Home” button works from all pages	Any page loaded	Clicking redirects to Home page
TC-09	Verify all navigation buttons are functional	Application deployed	All navigation buttons work without errors
TC-10	Verify Dashboard and KPI data load correctly	Dashboard/KPI page loaded	Data displays without broken elements